

Declare, Check, Derive: A Small Model Behind a Deterministic Judge, Measured on Screens

Rafael Uchôa

September 2026

Abstract

When a language model must produce an artifact that has to be right, one option is to have it write less. The model fills a short declaration over a closed vocabulary that a catalog enumerates. A deterministic check judges the declaration and quotes its errors back for repair. A function derives the artifact, and a test compares a projection of the artifact with an oracle computed from the declaration alone, without the artifact’s expensive runtime. We state the pattern, the four conditions a domain must meet for it to apply, and four claims it rests on, and we measure those claims on one domain where generated output is usually judged by eye: user interface screens. The instance is a view kit whose screens are an archetype with slots filled by declared use cases, with a fictional catalog of 38 use cases and twelve screens across four archetypes. Four experiments were declared in writing before their code. Under a JSON schema and at most three repair turns quoting the check, `gpt-oss-120b` at low reasoning effort declares a screen the check accepts on the first try 114 times in 120 and every time within one repair, for US\$0.40 per thousand screens, but agrees with the screen the author wrote by hand only 63 times in 120; every disagreement is a choice of which fields to show. Across 893 single mutations of the twelve reference screens, the check rejects 67%, and of the 298 it accepts all but two change the outline a reviewer would read. The outline is weaker against a wrong derivation: it kills 53% of 168 viable mutants of the derivation and archetype code, against a bar of 80%, and a golden recorded from the same code adds nothing. A case costs 0.42ms in process and 9.8ms through headless Chrome. We argue, without measuring, which of these findings should transfer to other domains that meet the conditions and which belong to screens.

1 Introduction

Language models are now asked for artifacts that must be right: code, configuration, queries, interfaces. The usual arrangement has the model write the artifact itself, and then judges it by running it, by a second model, or by similarity to a reference. Each of these is expensive, probabilistic or both, and none decides validity before the artifact exists.

We study the opposite arrangement, in which the model writes as little as possible. A *catalog* enumerates what exists. The model fills a short *declaration* over the catalog’s closed vocabulary. A *check* with a handful of deterministic rules accepts the declaration or returns errors precise enough to quote back to the model. A *derivation* turns an accepted declaration into the artifact, with no model involved. A test reads a *projection* of the artifact, the few facts it is meant to state, and compares it with an *oracle* that computes the same projection from the declaration alone, without the artifact’s runtime. The model’s freedom is confined to the declaration, and every step after it can be tested like ordinary code.

The arrangement is not specific to any artifact, but it can only be measured on one. We chose user interface screens. Interfaces derived from models of the application are an old idea [4, 22, 26].

Language models now write interfaces directly [3, 15, 25, 30], and that output is almost always judged by rendering and looking. If the arrangement holds on screens, where a check that fails is least expected, that is evidence for it. It is not proof for other domains, and we keep the two apart throughout.

The arrangement rests on four claims, each of which had been argued and none measured:

1. A small hosted model can write declarations the check accepts across the catalog, not only for the one case it was calibrated on.
2. The check rejects most wrong declarations, and a wrong declaration it accepts still changes the projection, so a reviewer reading the projection sees it.
3. The oracle is independent enough of the derivation that a wrong derivation fails some test.
4. Testing the projection in process is much cheaper than testing through the artifact’s runtime, and loses nothing where that runtime adds no meaning.

Contributions.

- The arrangement stated independently of any domain, with four conditions a domain must meet for it to apply (§3).
- One instance, a view kit whose screens are declarations over archetypes and use cases, and four experiments on it, each declared in writing with a yes-or-no signal before its code (§4, §5, §6).
- The finding that passing the check and agreeing with the author are different measurements. The configuration that passes most cheaply agrees least, and the disagreement falls entirely on a fact the catalog does not state (§6.1).
- A mutation study of the declaration that separates what the check decides from what it leaves to a reviewer, and shows the model’s own accepted answers fall in the second group (§6.2).
- A mutation study of the derivation that measures the oracle’s independence and shows that the projection, not the oracle, bounds what the test sees (§6.3).
- An argued account of which findings should transfer to other domains and which belong to screens (§7.2).

2 Related work

Model-based user interfaces. Deriving interfaces from task and domain models was studied long before language models; Szekely reviews the first generation [26], ConcurTaskTrees is a well-known task notation [22], the CAMELEON framework relates task, abstract and concrete interfaces across targets [4], and Myers, Hudson and Pausch survey why such tools did and did not reach practice [19]. Our declaration is close to what that literature calls an abstract user interface, restricted to a handful of archetypes, and the use case headers play the part of the domain model. Model-driven accessible interfaces have recently been generated with language models [10]. What we add is not a richer model but measurement of a small one with a check that fails.

Language models writing interfaces. pix2code generated interface code from screenshots [3]; Design2Code benchmarks conversion of web pages to code [25]; WebSight is a synthetic dataset for the same task [13]; UICoder fine-tunes on automated feedback from compilers and vision models [30]; and Leviathan et al. report language models generating whole interfaces in response to prompts [15]. All of these produce markup or code, judged by rendering. Our model produces a declaration small enough that a program can decide whether it is valid.

Structured output and repair. Constraining generation to a grammar or a schema is well studied [7, 29], and JSONSchemaBench measures how well engines and providers hold a schema [8]. Format restrictions can cost reasoning quality [27]. Feeding errors back for a new attempt appears as self-refinement [17], self-debugging from execution feedback [5], verbal reflection [24], and self-repair, whose gains depend on the quality of the feedback [21]. Our repair turn quotes a deterministic check verbatim, the most precise feedback available. Settings meant to be deterministic are not, in practice [1], which matches what we observed at temperature zero.

Oracles, mutation and browser tests. The oracle problem, deciding whether an output is correct, is surveyed by Barr et al. [2]; model-based testing, of which our oracle is an instance, by Utting, Pretschner and Legeard [28]. Mutation analysis originates with DeMillo, Lipton and Sayward [6]; Offutt et al. studied which operators suffice [20]; Jia and Harman survey the field [11]; and Just et al. found mutant detection correlates with real fault detection [12]. Browser tests are a known source of cost and flakiness [16, 18, 23], and recorded or programmed web tests break as pages change [9, 14]. We do not propose a new mutation method; we use a small one to test a claim of independence.

3 The pattern

The arrangement has five parts and one place for a model. The *catalog* enumerates the things a declaration may name and states the facts about each that the rest of the pipeline needs. A *declaration* is a short value over that vocabulary. The *check* is a total, deterministic function from a declaration to a list of errors, empty when the declaration is valid. The *derivation* is a deterministic function from a valid declaration to the artifact. The *projection* reads from the artifact the facts it is meant to state, and the *oracle* computes the same facts from the declaration and the catalog without calling the derivation. The model writes declarations and nothing else. A JSON schema generated from the catalog shapes its answer, and the check’s errors, quoted verbatim, are its repair feedback.

A domain admits the pattern when four conditions hold. These are ours, and argued.

1. **Closed vocabulary.** What a declaration may name can be enumerated from a catalog before the model is called.
2. **Decidable validity.** Whether a declaration is well formed can be decided by rules over the declaration and the catalog, without running the artifact.
3. **Deterministic derivation.** A valid declaration determines the artifact, so that everything the model did not write can be tested as ordinary code.
4. **A projection with an independent oracle.** The facts the artifact is meant to state can be read from it and computed from the declaration by separate code.

Screens meet all four when a view kit closes its vocabulary. So do several other artifacts, by the same argument and without measurement. In declarative infrastructure, the catalog is the available resource types and the projection is the resource graph. In an API contract, the catalog is the operations and types and the projection is the routes with their payload types. In an access policy, the catalog is roles and resources and the projection is the matrix of who may do what. In a workflow, the catalog is the steps and events and the projection is the reachable transitions. Where the vocabulary cannot be closed, as in free-form code, the pattern does not apply, and nothing here speaks to that case.

4 The instance: screens

The instance is a Ruby view kit, written in Phlex, that closes its vocabulary. Components take explicit keywords, class names live only in a theme, and a page renders exactly one *archetype* whose *slots* are the only places content goes. In terms of the pattern, the catalog is a set of use case headers and archetypes, a declaration is a screen, the artifact is HTML, and the projection is a plain-text *outline* of the page.

4.1 The catalog and the declaration

A *use case header* states the facts the kit needs about one application operation, and nothing it would need to run it: a name, a kind (query or command), for a query its *shape* (collection, one, aggregate or events), the record type, what it accepts and returns with a type per field (text, money, enum, date, list, or a reference to a record), what a command requires, and which queries it invalidates. The kit reads headers; it never calls the operation.

An archetype declares its slots, each with the shapes it accepts, whether it is required and whether it repeats. Four exist: `master_detail` (a required collection `master`, a one-record `detail`); `list_with_feed` (a required collection `list`, an events `feed`); `dashboard` (a command `filter`, a required aggregate `kpis`, repeating `sections`); and `settings_sheet` (repeating command `areas`).

A screen declaration names one archetype and fills its slots:

```
screen :books, archetype: :master_detail do
  master "Books::ListBooks", columns: %i[title author isbn status shelf],
    omit: %i[cataloguer_note]
  detail "Books::ShowBook", columns: %i[title author isbn status publisher copies],
    omit: [], inputs: {id: selection(:master)}
  action "Books::WithdrawBook"
end
```

Every returned field must be placed in `columns` or `omit`; every accepted input has a source, a `selection` of another slot, a screen `filter`, or a `param` of the address. A command is placed as an `action`, naming only the use case; where its button goes is derived.

4.2 The check

Seven rules, in order: the slot exists in the archetype; the use case exists and is placed once; a command is an action (or fills a slot shaped for commands) and some slot shows a record of its type; a query's shape fits its slot; every returned field is in exactly one of `columns` and `omit` and no other field appears; every input is supplied and a `selection` names a collection slot of the right

record type; every required slot is filled. Each error is a string naming the slot, the use case and what is wrong, so that it can be quoted back to a model.

4.3 Derivation, outline and oracle

The derivation maps each shape to a kit component: a collection or events to a table, one or aggregate to a property list, a command to a toolbar button or, in a settings area, a form built from what it accepts. The page it returns takes the results the host fetched and nothing about how.

The *outline* is read from rendered HTML by element and role, never by class name: one line per fact, such as `master columns: Title | Author, master rows: 3, master selected: 2, actions: Withdraw book`. The *oracle* computes the same outline from the declaration, the headers and the seed data, and shares no code with the derivation. A screen’s test asserts that the outline of the page equals the oracle’s, and equals a committed *golden*. The design argues that this replaces a browser test per screen, because the kit keeps scripts inside components and a screen’s meaning is in its markup.

4.4 Generation

A model is given a fixed system prompt stating the rules above in prose, a user message generated from the catalog (the screen’s goal in a person’s words, one line per archetype with its slots, one line per offered use case with its header, and one worked example from another domain), and a JSON schema generated from the same catalog, with one branch per offered use case that pins its name and limits its fields. The answer is rendered into a declaration and checked. If the check fails, a repair turn quotes its errors verbatim and asks for the whole answer again, at most three times. The model is never told the archetype.

5 Experimental setup

The catalog. A fictional lending library, written for this study before any experiment ran: 38 use case headers over books, members, loans, holds, fines, purchase orders, events, suppliers, two dashboards and two settings areas, and twelve reference screens written by hand, five master-details, three lists with a feed, two dashboards and two settings sheets. Every reference screen passes the check. Each screen has one sentence of goal, naming records and actions, not fields. Internal fields are marked only by their names (`desk_note`, `buyer_note`, `account_ref`).

Seeds. For each screen, synthetic records are built from the return types: empty, one record, three records, three records with 200-character strings, and, where the screen has a selection source, three records with the second selected and three with a selection that matches nothing. This gives 58 cases, the *lab oracle suite*.

Models. `openai/gpt-oss-120b` at reasoning effort low was the configuration under test, chosen from an earlier one-screen calibration. `openai/gpt-oss-20b` at low and `qwen/qwen3.6-27b` at its default effort were recorded for comparison. All through Groq’s OpenAI-compatible chat completions endpoint, with `response_format` set to a JSON schema without `strict`, and the provider’s default temperature, because temperature zero had not been deterministic in calibration. Ten runs per screen per configuration. Prices are the per-million-token rates Groq listed on 2026-09-22; the rate for `qwen3.6-27b` was not listed and is assumed equal to a sibling model’s.

Experiment	Signal	Measured	Answer
Generation, twelve screens	$\geq 108/120$ first, $\geq 118/120$ final	114/120, 120/120	yes
Declarations the check lets through	$\geq 60\%$ rejected, none invisible	66.6%, 2 invisible	no
A wrong derivation against the oracle	$\geq 80\%$ of viable killed	53.0%	no
A screen without a browser	median ≤ 5 ms, ratio ≥ 20	0.42 ms, 23.6	yes

Table 1: The four declared experiments. All were run on 2026-09-22.

Configuration	First try	Final	Same screen	Ignoring order	Tokens in/out	US\$/1k
<code>gpt-oss-120b</code> , low	114/120	120/120	54/120	63/120	1,349 / 334	0.40
<code>gpt-oss-20b</code> , low	83/120	119/120	49/120	56/120	1,765 / 322	0.23
<code>qwen3.6-27b</code> , default	118/120	120/120	106/120	113/120	1,222 / 778	4.09*

Table 2: Twelve screens, ten runs each. *Final* is within three repair turns. *Same screen* compares the final answer with the hand-written reference; tokens are per screen, repairs included. *Assumed rate.

Mutation. Declarations were mutated on the parsed screen values by sixteen operators (Table 3). The derivation and the four archetype classes were mutated in their source by six operators: deleting a statement, negating a condition, flipping a boolean, emptying a string, replacing a keyword argument’s value with `nil`, and swapping `first` and `last`. Each source mutant ran in its own process against a copy of the library.

Timing. On an Apple M3 Pro with Ruby 4.0.7, one Rack application served every case. In process, through a mock request; in a browser, through headless Chrome 153 driven by Capybara and Cuprite, against the same application on a local port. One warm-up pass, then five timed passes per transport.

Declared signals. Every experiment was declared in writing, with a hypothesis, a yes-or-no signal and its thresholds, before its code was written, and no threshold moved after a run. The thresholds are ours and are argued in each record: 90% first-try and 98% final passes for generation; 60% rejection and no invisible accepted mutant for the check; 80% of viable mutants killed for the oracle; a 5 ms median case and a browser at least twenty times slower for the transport.

6 Results

Table 1 summarises the four signals. Two passed and two did not. Before any mutant was built, the unmutated derivation failed two cases of the lab oracle suite: a dashboard with no aggregate result rendered its labels with empty values, where the oracle says the slot shows nothing. The derivation was fixed and a test added; every number below is on the fixed code.

6.1 Generation passes the check and disagrees with the author

Table 2 gives the results. The configuration under test cleared its signal: 114 of 120 answers passed the check on the first try and all 120 within one repair. First-try passes by archetype were 47/50 for master-detail, 29/30 for list with feed, 18/20 for dashboard and 20/20 for settings sheet. The 412 calls of the three configurations cost US\$0.57; no call was rejected by the provider or rate-limited.

Operator	Mutants	Rejected	Visible	Invisible
forget field, unknown field, duplicate	159	159	0	0
move slot, wrong archetype, command in slot	86	86	0	0
drop input, source slot	12	12	0	0
foreign action	176	170	6	0
foreign use case	223	156	67	0
drop placement	27	10	17	0
source kind	14	2	10	2
column to omit, omit to column, reorder	185	0	185	0
drop action	11	0	11	0
total	893	595 (66.6%)	296	2

Table 3: Every single mutation of the twelve reference screens. *Visible* and *invisible* count accepted mutants whose outline, for the declared seed, does or does not differ from the reference’s.

Passing the check is not agreeing with the author. The same configuration declared the reference screen, ignoring column order, 63 times in 120, and on two screens never. The larger **qwen3.6-27b** passed as often and agreed 113 times, at about ten times the assumed price. Of the 155 accepted answers that differ from their reference across the three models, 149 differ only in which fields are in **columns** and which in **omit**, or in their order; five give a command an input object the declaration language cannot hold, and are identical once rendered; one declares a screen under another archetype the check also accepts. The models showed internal fields such as **buyer_note** and hid real ones such as a book’s **shelf**. Nothing in a header marks a field internal except its name.

Column order, which the earlier one-screen calibration had singled out as the remaining source of disagreement, is the smaller part here: ignoring it recovers 9 of the 66 disagreements of **gpt-oss-120b**.

Without **strict**, the provider returned answers the schema forbids: a misspelled use case (**Fines::WaaveFine**), a name with a space inserted, fields of one use case placed on another. The check caught every one, which is the sense in which the check, and not the schema, is the judge.

6.2 What the check decides and what it leaves

Table 3 groups the sixteen operators by outcome. The check divides them almost cleanly. Structural mistakes, a field forgotten or invented, a use case in a slot of the wrong shape, a missing input, are rejected every time. Choices, which fields show and in what order, and whether a command is offered, are never rejected. The first half of the signal passed.

The second half did not, by two mutants. On the acquisitions dashboard, an input read from the address, **param(month)**, became **filter(month)**. The check accepts any source for an input that is not a selection, and the outline has no line saying where an input comes from. The page is different in practice and identical in its outline.

The accepted mutants that are visible matter as much. A settings area holding the command that withdraws a book, instead of the loan-period setting, passes the check, because an area accepts any command and a settings sheet shows no record for the command to need. Only the outline shows it, where the area’s one field reads **Reason** instead of **Days**. The check cannot know which screen a command belongs to, because the catalog does not say.

Set against generation, the model’s accepted answers fall entirely inside the operators the check accepts by design. All 150 of the 155 accepted model differences that are real change the outline.

Operator	Viable	Killed by oracle suite	Killed by gem suite	Survived oracle
delete statement	67	42	51	25
negate condition	10	8	9	2
flip boolean	7	2	5	5
empty string	25	5	10	20
nil keyword value	58	32	44	26
first and last	1	0	0	1
total	168	89 (53.0%)	119 (70.8%)	79

Table 4: Mutants of the derivation and the four archetype classes. Fifteen further mutants did not load and are excluded. The *gem suite* is the kit’s own unit and snapshot tests.

No model answer was a wrong declaration a reviewer of outlines could not see.

6.3 How independent the oracle is

Table 4 shows the oracle suite killing 89 of 168 viable mutants, well under the declared 80%. The kit’s own unit and snapshot tests kill 119, and the two together 122 (72.6%). The suites overlap less than expected: the unit tests kill 33 mutants the oracle misses, mostly links, element ids, the back button and tab state, while the oracle kills three the unit tests miss.

The golden killed nothing the oracle did not, and could not have. The golden is an outline recorded from the unmutated code, where the outline equals the oracle’s; any mutant that matches the oracle matches the golden. A golden guards a change across commits, not a derivation that was already wrong when the golden was recorded. This was not foreseen when the experiment was declared.

Reading the 79 survivors one by one, which is a reading and not a measurement: 22 change what the outline deliberately ignores (links, class names, the names of script actions); 23 are equivalent; 25 change behaviour on inputs no lab screen exercises, such as a dashboard filter or a date field in a settings form; and 9 change something the outline is meant to show and does not. Among the nine are a settings area losing its heading, every tab panel of a settings sheet rendered hidden, and enum fields rendered as text inputs, because the outline reads a form’s labels and not its controls.

6.4 What the browser costs

In process, the median case took 0.416 ms (90th percentile 0.704 ms), and a pass over all 58 cases 25.4 ms. Through headless Chrome, already started, the median case took 9.821 ms (16.267 ms), and a pass 683.1 ms; starting Chrome took a further 372 ms. The ratio of medians is 23.6, above the bar of twenty, and far below the orders of magnitude the design had argued. Both transports read identical outlines for all 58 cases.

7 Discussion

7.1 On screens

The measurements divide the pipeline’s work more sharply than the design had. The check is a good judge of structure and no judge of taste: it rejects every structural mutation and accepts every choice of fields, and the language model’s disagreements with the author are exclusively choices of fields. Whether a model “declares the right screen” therefore depends on something the catalog

does not state. The narrowest repair is a header flag marking a field internal, so that which fields show is derived rather than chosen; these runs are the first evidence that such a flag would remove most of the disagreement, and they do not measure it.

The same division explains why a cheaper model can be the right one. The configuration under test passes the check as often as the larger model and costs a tenth as much; what it loses is agreement on field choices, which a reviewer sees in the outline and which a header flag would take away from the model. Choosing a model by agreement with one author’s taste would be choosing by the least stable quantity measured here.

The oracle is independent in the sense that matters for correctness of what the outline reads, and blind outside it. Most survivors change links, classes or script wiring the outline was designed to ignore, or code no lab screen exercises. The nine that matter share a cause: the outline reads what a screen *says*, and not what kind of control says it or whether it is visible. Both are recoverable from HTML by role and attribute, without class names, and adding them is a change to the outline’s contract, not to the design. The golden’s null contribution is a cleaner lesson: a snapshot recorded from the code under test cannot catch a bug that code had when the snapshot was taken, only one introduced later, which is what a golden is for and all it is for.

The browser result weakens the design’s cost argument and leaves its structural argument standing. Nine milliseconds per case is not a cost teams reorganise suites around. What the in-process test removes is the browser as a dependency: no binary, no port, no second process to be flaky in. That is the argument the measurements support.

7.2 What should transfer

Everything in this subsection is argued from one domain. Each finding is restated in the terms of the pattern, with the measurement it comes from and the reason we expect it to hold wherever the four conditions do. None of the reasons depends on what a screen is.

Validity and agreement are different quantities. On screens, the check accepted 120 of 120 declarations while the model matched the author’s 63 times. The check decides what the catalog states. It cannot decide what the catalog leaves to taste, and a model’s disagreements will concentrate there, because that is the only place it is free. The general remedy is to move the fact into the catalog, such as a field marked internal, a resource marked public, or a route marked deprecated. Buying a larger model to guess it is the alternative. What a model gets “wrong” after passing a good check is a map of what the catalog does not say.

Behind a precise check, the small model is the right default. The cheapest configuration passed as often as one costing about ten times more, and needed at most one repair. A check whose errors can be quoted verbatim is the most precise feedback a repair loop can have [21], and it narrows the gap between models to what the check does not judge. Choosing a model by agreement with one author would be choosing by the least stable quantity measured here.

The check, not the schema, is the judge. Without strict decoding, the provider returned names the schema forbids, and the check caught every one. Schema support varies by engine and provider [8], and a pipeline that trusts it inherits that variance. A deterministic check makes the schema a hint rather than a guarantee, in any domain.

The projection, not the oracle, bounds the test. The survivors of derivation mutants that mattered changed facts the outline does not read. An oracle can only be as independent as the projection it is compared through. In another domain, the question to ask of the test is not whether the oracle shares code with the derivation. It is which facts of the artifact the projection leaves out.

A golden recorded from the code under test guards change, not correctness. The golden added no kill to the oracle, by construction. Any snapshot recorded from the code it tests agrees with that code’s existing bugs. This is known in principle and easy to forget in practice, and it holds for snapshots of anything.

Removing the runtime buys independence more than speed. The browser cost a factor of 24, one order of magnitude rather than several. What the in-process test removes is a second process, a binary and a source of flakiness [16, 23]. We expect the same trade for any artifact whose runtime is a database, a cloud account or a queue, and the ratio to depend on that runtime.

Two findings should not transfer as they stand. The rates themselves, 95% on the first try, 67% rejected and 53% killed, belong to this catalog, this check and this outline. The specific blind spots, control kinds and panel visibility, belong to HTML.

8 Limitations

- **One domain.** The pattern is stated generally and measured once. The conditions of §3, the examples of other domains and every claim of §7.2 are argued. A second instance, in a domain whose runtime is not a browser, is the measurement they wait for.
- **One catalog of twelve screens,** fictional, written by the author in one sitting, with goals written by the same person as the references. Agreement is agreement with that person.
- **One provider and three models,** ten runs per screen, one day. The ranking of models may not survive another provider, another day or another catalog. One model’s price is assumed.
- **The mutation operators are ours** and small subsets of what mutation tools apply. A mistake no operator makes is unmeasured; the model’s accepted answers were checked against the declaration operators, and all were explained, but no such check exists for the derivation.
- **One seed per screen decides visibility** in the declaration study. An accepted mutation that changes the outline only for another seed counts as invisible.
- **Survivor readings are readings.** The four categories of derivation survivors, one of them added after the run, are judgements of one person reading diffs.
- **No script runs in any timed case.** Where a screen’s meaning depends on a script, the browser comparison does not apply.
- **No person used a generated screen.** Everything measured is about declarations and markup; nothing here says a generated screen serves its goal.

9 Conclusion

A language model can be confined to writing a declaration over a closed vocabulary, with a deterministic check as its judge, a derivation doing the rest and a projection tested against an independent oracle. Each step can be measured with a signal fixed in advance. On screens, the one domain measured, a small hosted model almost always writes a declaration the check accepts, cheaply, and half the time writes the one its author would. The check catches every structural mistake and none of the choices. The projection shows all but two accepted mistakes of declaration but only half the mistakes of derivation. The runtime it replaces costs nine milliseconds a case. The two lessons we expect to outlive the domain are these. What a model disagrees on after passing the check is what the catalog should state. What the test misses is what the projection does not read.

Reproducibility. Every experiment’s declaration, results and marked doubts, the catalog, the goals, the harnesses, the exact prompt and schema, one record per model call with its raw answer and token usage, every mutant with its verdict, and every timing are kept in the accompanying repository under `docs/experiments/` and `lab/`. No threshold moved after a run.

References

- [1] Berk Atil, Sarp Aykent, Alexa Chittams, Lisheng Fu, Rebecca J. Passonneau, Evan Radcliffe, Guru Rajan Rajagopal, Adam Sloan, Tomasz Tudrej, Ferhan Ture, Zhe Wu, Lixinyu Xu, and Breck Baldwin. Non-determinism of “deterministic” LLM settings, 2024.
- [2] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5): 507–525, 2015. doi: 10.1109/TSE.2014.2372785.
- [3] Tony Beltramelli. pix2code: Generating code from a graphical user interface screenshot, 2017.
- [4] Gaëlle Calvary, Joëlle Coutaz, David Thevenin, Quentin Limbourg, Laurent Bouillon, and Jean Vanderdonckt. A unifying reference framework for multi-target user interfaces. *Interacting with Computers*, 15(3):289–308, 2003. doi: 10.1016/S0953-5438(03)00010-9.
- [5] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug, 2023.
- [6] R. A. DeMillo, R. J. Lipton, and F. G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.
- [7] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning, 2023.
- [8] Saibo Geng, Hudson Cooper, Michał Moskal, Samuel Jenkins, Julian Berman, Nathan Ranchin, Robert West, Eric Horvitz, and Harsha Nori. JSONSchemaBench: A rigorous benchmark of structured outputs for language models, 2025.
- [9] Mouna Hammoudi, Gregg Rothermel, and Paolo Tonella. Why do record/replay tests of web applications break? In *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pages 180–190, 2016. doi: 10.1109/ICST.2016.16.

- [10] Blessing Jerry, Lourdes Moreno, Virginia Francisco, and Raquel Hervás. LLM-driven accessible interface: A model-based approach, 2026.
- [11] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.
- [12] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. Are mutants a valid substitute for real faults in software testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 654–665, 2014. doi: 10.1145/2635868.2635929.
- [13] Hugo Laurençon, Léo Tronchon, and Victor Sanh. Unlocking the conversion of web screenshots into HTML code with the WebSight dataset, 2024.
- [14] Maurizio Leotta, Diego Clerissi, Filippo Ricca, and Paolo Tonella. Capture-replay vs. programmable web testing: An empirical assessment during test case evolution. In *2013 20th Working Conference on Reverse Engineering (WCRE)*, pages 272–281, 2013. doi: 10.1109/WCRE.2013.6671302.
- [15] Yaniv Leviathan, Dani Valevski, Matan Kalman, Danny Lumen, Eyal Segalis, Eyal Molad, Shlomi Pasternak, Vishnu Natchu, Valerie Nygaard, Srinivasan Venkatachary, James Manyika, and Yossi Matias. Generative UI: LLMs are effective UI generators, 2026.
- [16] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. An empirical analysis of flaky tests. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pages 643–653, 2014. doi: 10.1145/2635868.2635920.
- [17] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegraffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023.
- [18] A. M. Memon. GUI testing: Pitfalls and process. *Computer*, 35(8):87–88, 2002. doi: 10.1109/MC.2002.1023795.
- [19] Brad Myers, Scott E. Hudson, and Randy Pausch. Past, present, and future of user interface software tools. *ACM Transactions on Computer-Human Interaction*, 7(1):3–28, 2000. doi: 10.1145/344949.344959.
- [20] A. Jefferson Offutt, Ammei Lee, Gregg Rothermel, Roland H. Untch, and Christian Zapf. An experimental determination of sufficient mutant operators. *ACM Transactions on Software Engineering and Methodology*, 5(2):99–118, 1996. doi: 10.1145/227607.227610.
- [21] Theo X. Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation?, 2023.
- [22] F. Paternò, C. Mancini, and S. Meniconi. ConcurTaskTrees: A diagrammatic notation for specifying task models. In *Human-Computer Interaction INTERACT '97*, pages 362–369. Springer US, 1997. doi: 10.1007/978-0-387-35175-9_58.
- [23] Alan Romano, Zihe Song, Sampath Grandhi, Wei Yang, and Weihang Wang. An empirical analysis of UI-based flaky tests. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1585–1597, 2021. doi: 10.1109/ICSE43902.2021.00141.

- [24] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023.
- [25] Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. Design2Code: Benchmarking multimodal code generation for automated front-end engineering, 2024.
- [26] Pedro Szekely. Retrospective and challenges for model-based interface development. In *Design, Specification and Verification of Interactive Systems '96*, Eurographics, pages 1–27. Springer Vienna, 1996. doi: 10.1007/978-3-7091-7491-3_1.
- [27] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models, 2024.
- [28] Mark Utting, Alexander Pretschner, and Bruno Legeard. A taxonomy of model-based testing approaches. *Software Testing, Verification and Reliability*, 22(5):297–312, 2011. doi: 10.1002/stvr.456.
- [29] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models, 2023.
- [30] Jason Wu, Eldon Schoop, Alan Leung, Titus Barik, Jeffrey P. Bigham, and Jeffrey Nichols. UICoder: Finetuning large language models to generate user interface code through automated feedback, 2024.